

Università degli Studi di Napoli Parthenope

Dipartimento di Ingegneria



***Corso di Laurea in Ingegneria e Scienze
Informatiche per la Cybersecurity***

Esame di Basi di Dati

**Progettazione e implementazione di un sistema informativo
per la gestione di una concessionaria automobilistica multi-sede**

AutoGarage

Docente

Daniele Granata

Studente

Stefano Vanzini

0334000208

Anno accademico 2025-2026

Indice

1	Traccia d'Esame - Progetto di Sistema Informativo per la gestione di una concessionaria automobilistica multi-sede	3
2	Obiettivi del Progetto	5
3	Analisi e progettazione concettuale: Modello Entità - Relazione	6
3.1	Modello E-R Base	6
3.2	Risoluzione della generalizzazione Dipendente – Venditore/Meccanico.....	8
3.2.1	Scenario 1 – Accorpamento al padre.....	8
3.2.2	Scenario 2 – Accorpamento alle figlie	9
3.2.3	Scelta adottata	10
3.3	Modello E-R Intermedio	11
3.4	Risoluzione della generalizzazione Utente – Cliente/Dipendente	12
3.4.1	Scenario 1 – Accorpamento al padre.....	12
3.4.2	Scenario 2 – Accorpamento alle figlie	13
3.4.3	Scelta adottata	14
3.5	Modello E-R Finale.....	15
3.6	Analisi delle cardinalità.....	17
4	Progettazione Logica.....	19
4.1	Struttura tabelle	20
4.1.1	Utente	20
4.1.2	Auto.....	21
4.1.3	Sede	22
4.1.4	Fornitore	22
4.1.5	Recensione	23
4.1.6	TestDrive	23
4.1.7	Vendita.....	24
4.1.8	Noleggio	24
4.1.9	Intervento	25
4.1.10	Fornitura	26
4.1.11	Impiego.....	26
4.2	Vincoli interrelazionali	27
4.3	Trigger implementati	29

4.3.1	Trigger before_insert_vendita	29
4.3.2	Trigger after_insert_vendita	29
4.3.3	Trigger before_insert_noleggio	29
4.3.4	Trigger before_insert_testdrive	30
4.3.5	Trigger before_insert_intervento.....	30
4.3.6	Trigger before_insert_recensione.....	30
4.3.7	Trigger before_insert_impiego.....	31
5	Implementazione del sistema informativo	32
5.1	Login	32
5.2	Registrazione.....	33
5.3	Catalogo Auto.....	33
5.4	Dettaglio Auto	34
5.5	Prenotazione Test Drive	35
5.6	Ricerca auto per marca	35
5.7	Visualizzazione recensioni	36
5.8	Dashboard venditore	36
5.9	Dashboard meccanico	37
6	Gestione delle sessioni	38
7	Attacchi e contromisure	39
7.1	SQL Injection	39
7.2	Attacco a forza bruta	40
7.3	Attacco a dizionario	42
8	Conclusioni.....	43

1 Traccia d'Esame - Progetto di Sistema Informativo per la gestione di una concessionaria automobilistica multi-sede

Il progetto si propone di sviluppare un sistema informativo con l'obiettivo di digitalizzare e ottimizzare la gestione delle attività svolte all'interno di una concessionaria automobilistica.

Il sistema AutoGarage viene sviluppato come applicazione web full-stack accessibile tramite browser da qualsiasi dispositivo dotato di connessione Internet. L'interfaccia è progettata per garantire semplicità d'utilizzo, accessibilità e rapidità nella consultazione delle informazioni.

Il cuore del sistema è costituito da un database relazionale MySQL progettato secondo i principi della progettazione concettuale, logica e fisica. Il database è stato realizzato con particolare attenzione ai vincoli di integrità referenziale, alla consistenza dei dati e all'efficienza delle operazioni di inserimento, modifica e consultazione.

Le principali entità coinvolte nel sistema sono:

- **Utente:** rappresenta le persone che interagiscono con il sistema. Gli utenti possono assumere il ruolo di Cliente, Venditore o Meccanico.
- **Auto:** contiene tutte le informazioni relative ai veicoli presenti nel catalogo della concessionaria, come marca, modello, anno di immatricolazione, alimentazione, prezzo, stato e sede di appartenenza.
- **Sede:** identifica le diverse filiali della concessionaria e consente di distribuire il parco auto sul territorio.
- **Test Drive:** gestisce le prenotazioni effettuate dai clienti per provare un determinato veicolo.
- **Recensione:** permette ai clienti di esprimere una valutazione sui veicoli presenti nel catalogo.
- **Vendita:** registra le operazioni di acquisto dei veicoli effettuate dai clienti.
- **Noleggio:** consente la gestione dei contratti di noleggio associati alle automobili disponibili.
- **Intervento:** tiene traccia delle attività di manutenzione e riparazione effettuate dai meccanici sui veicoli.

- **Fornitore e Fornitura:** permettono di monitorare la provenienza dei veicoli e le operazioni di approvvigionamento.

L'applicativo fornirà diverse funzionalità tra cui:

- Registrazione e autenticazione degli utenti.
- Visualizzazione del catalogo delle automobili disponibili.
- Consultazione della scheda dettagliata di ciascun veicolo.
- Ricerca e filtraggio delle automobili per marca.
- Prenotazione di test drive da parte dei clienti.
- Visualizzazione delle recensioni associate ai veicoli.
- Dashboard dedicata ai venditori.
- Dashboard dedicata ai meccanici.
- Gestione dei dati attraverso un sistema di ruoli e permessi.

L'intera applicazione è stata progettata seguendo un'architettura modulare che favorisce la manutenibilità del codice e la possibilità di introdurre nuove funzionalità in futuro. Tra le possibili evoluzioni del sistema rientrano l'integrazione di sistemi di pagamento online, la gestione automatizzata dei contratti di vendita e noleggio, l'invio di notifiche ai clienti e la generazione di report statistici sulle attività della concessionaria.

1. Clienti (Utenti standard)

- Consultazione catalogo auto.
- Visualizzazione schede dettagliate.
- Ricerca per marca.
- Prenotazione test drive.
- Consultazione recensioni.

2. Venditori (Utenti avanzati)

- Accesso alla dashboard venditore.
- Visualizzazione prenotazioni test drive.

3. Meccanici (Utenti avanzati)

- Accesso alla dashboard meccanico.
- Visualizzazione degli interventi assegnati.

2 Obiettivi del Progetto

Il settore automobilistico rappresenta uno dei comparti economici più rilevanti a livello nazionale e internazionale. Le concessionarie si trovano quotidianamente a gestire un elevato volume di informazioni relative ai veicoli disponibili, ai clienti, alle attività commerciali e agli interventi di manutenzione. In questo contesto, la digitalizzazione dei processi rappresenta un elemento fondamentale per migliorare l'efficienza operativa e la qualità dei servizi offerti.

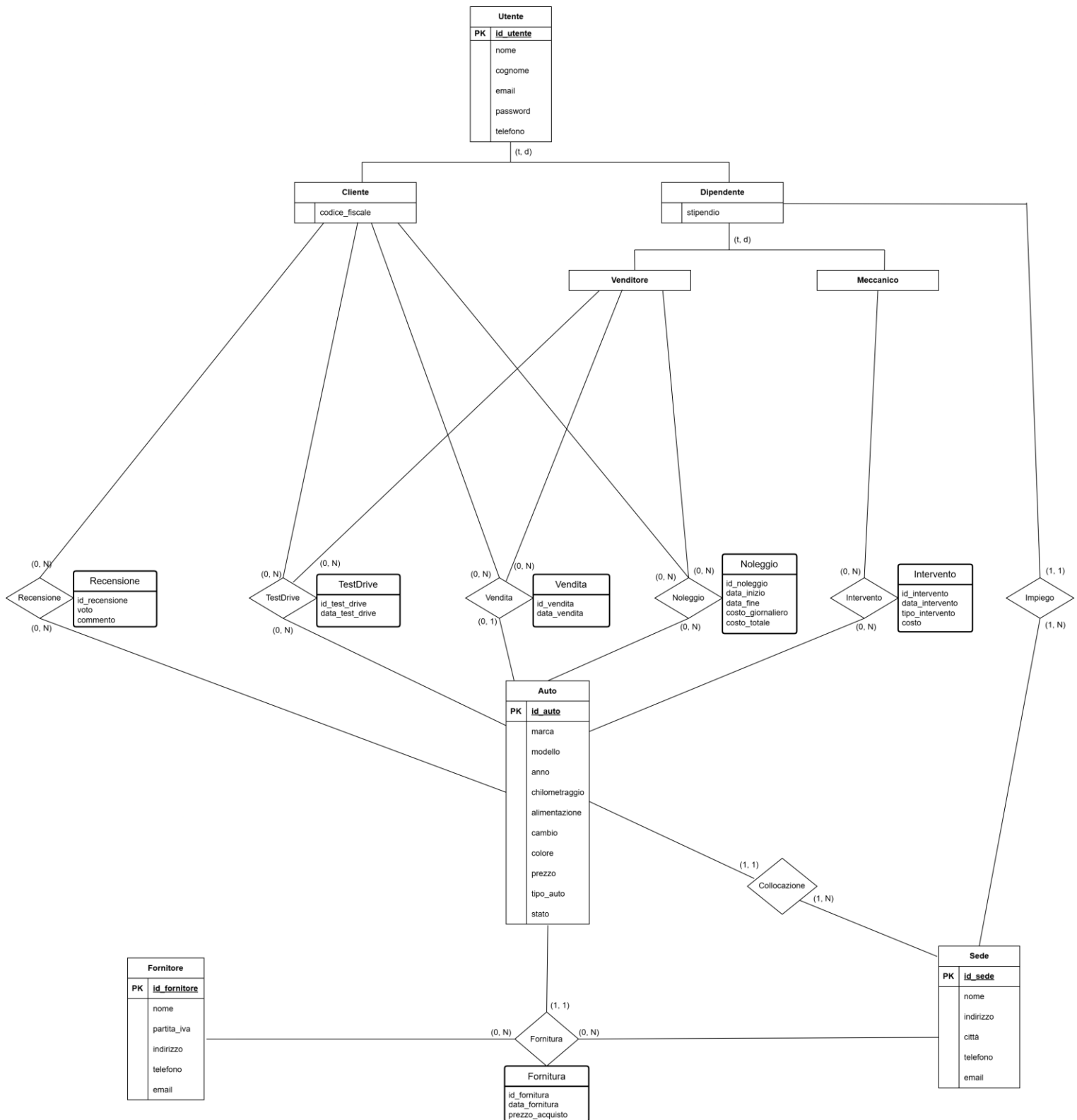
Alla luce di queste esigenze, il sistema AutoGarage si propone di fornire una piattaforma centralizzata per la gestione delle principali attività di una concessionaria automobilistica multi-sede. Il sistema è pensato per adattarsi sia a realtà di piccole dimensioni sia a concessionarie più strutturate, caratterizzate dalla presenza di più sedi operative e di diverse figure professionali coinvolte nei processi aziendali.

L'obiettivo principale del progetto è quello di semplificare la gestione delle informazioni relative ai veicoli e agli utenti, consentendo una consultazione rapida e organizzata dei dati. Parallelamente, il sistema mira a migliorare l'esperienza dei clienti attraverso strumenti che permettono la consultazione del catalogo automobilistico, la visualizzazione delle caratteristiche dei veicoli e la prenotazione dei test drive.

Dal punto di vista organizzativo, la piattaforma fornisce strumenti dedicati ai dipendenti della concessionaria, permettendo ai venditori e ai meccanici di accedere a funzionalità specifiche in base al proprio ruolo. Tale approccio contribuisce a migliorare il coordinamento delle attività interne e a garantire una gestione più efficiente delle risorse aziendali.

3 Analisi e progettazione concettuale: Modello Entità - Relazione

3.1 Modello E-R Base



Il primo modello E-R realizzato per il sistema AutoGarage rappresenta la struttura concettuale iniziale della base di dati. In questa fase sono state individuate le principali entità del dominio applicativo, le relazioni tra esse e le cardinalità che descrivono i vincoli di partecipazione.

Il modello rappresenta una concessionaria automobilistica multi-sede, nella quale sono gestiti utenti, automobili, sedi, fornitori, test drive, vendite, noleggi, interventi di officina e recensioni.

Nel modello base sono presenti le seguenti entità principali:

- **Utente**, che contiene le informazioni comuni a tutti gli utenti del sistema, come nome, cognome, email, password e telefono;
- **Cliente**, specializzazione di Utente, caratterizzata dal codice fiscale;
- **Dipendente**, specializzazione di Utente, caratterizzata dallo stipendio;
- **Venditore**, specializzazione di Dipendente, incaricata della gestione di test drive, vendite e noleggi;
- **Meccanico**, specializzazione di Dipendente, incaricata della gestione degli interventi sui veicoli;
- **Auto**, che rappresenta i veicoli presenti nel catalogo della concessionaria;
- **Sede**, che rappresenta le diverse filiali della concessionaria;
- **Fornitore**, che rappresenta i soggetti da cui provengono le automobili.

Nel modello sono inoltre presenti diverse relazioni dotate di attributi propri. La relazione **TestDrive** collega cliente, venditore e auto, e memorizza la data della prova. La relazione **Vendita** collega cliente, venditore e auto, registrando la data di vendita. La relazione **Noleggio** collega cliente, venditore e auto, memorizzando data di inizio, data di fine, costo giornaliero e costo totale. La relazione **Intervento** collega auto e meccanico, registrando data, tipo di intervento e costo. La relazione **Recensione** collega cliente e auto, permettendo di memorizzare voto e commento. Infine, la relazione **Fornitura** collega fornitore, auto e sede, memorizzando data della fornitura e prezzo di acquisto.

Una caratteristica importante del modello base è la presenza di due generalizzazioni/specializzazioni.

La prima riguarda l'entità **Utente**, specializzata in **Cliente** e **Dipendente**.

La seconda riguarda l'entità **Dipendente**, specializzata a sua volta in **Venditore** e **Meccanico**. Entrambe le generalizzazioni sono state modellate come totali e disgiunte.

La generalizzazione **Utente** → **Cliente/Dipendente** è totale perché ogni utente del sistema deve appartenere obbligatoriamente a una delle due categorie previste. È disgiunta perché un utente non può essere contemporaneamente cliente e dipendente.

La generalizzazione **Dipendente** → **Venditore/Meccanico** è anch'essa totale perché ogni dipendente deve svolgere uno dei ruoli lavorativi previsti dalla concessionaria. È disgiunta perché un dipendente non può essere contemporaneamente venditore e meccanico.

Questo modello consente di rappresentare in modo chiaro la separazione concettuale tra clienti e personale interno, distinguendo ulteriormente i dipendenti in base al ruolo operativo svolto all'interno della concessionaria.

3.2 Risoluzione della generalizzazione Dipendente – Venditore/Meccanico.

3.2.1 Scenario 1 – Accorpamento al padre

La prima soluzione consiste nell'accorpare le entità figlie **Venditore** e **Meccanico** all'interno dell'entità padre **Dipendente**. In questo caso le entità Venditore e Meccanico vengono eliminate e viene introdotto nell'entità Dipendente un attributo discriminante, ad esempio **ruolo**, che può assumere i valori "venditore" e "meccanico".

Lo schema concettuale viene quindi semplificato mantenendo una sola entità Dipendente, contenente gli attributi comuni a tutti i dipendenti e il ruolo lavorativo svolto.

Vantaggi:

- riduzione del numero di entità e, successivamente, del numero di tabelle nel modello logico;
- maggiore semplicità nella gestione dei dipendenti;
- assenza di ridondanza degli attributi comuni;
- query più semplici quando si vuole ottenere l'elenco completo dei dipendenti;
- maggiore flessibilità nel caso in cui in futuro si vogliano aggiungere nuovi ruoli lavorativi.

Svantaggi:

- la distinzione tra venditore e meccanico non è più rappresentata tramite entità separate;
- alcuni vincoli semantici devono essere controllati tramite l'attributo ruolo;
- il database deve garantire che solo i venditori possano essere associati a vendite, noleggi e test drive;
- il database deve garantire che solo i meccanici possano essere associati agli interventi.

Questa soluzione è adatta quando le sottoclassi non possiedono attributi specifici rilevanti e si distinguono principalmente per il ruolo operativo svolto nel sistema.

3.2.2 Scenario 2 – Accorpamento alle figlie

La seconda soluzione consiste nell'eliminare l'entità padre **Dipendente** e mantenere soltanto le entità figlie **Venditore** e **Meccanico**. In questo caso gli attributi comuni del dipendente, come lo stipendio, verrebbero duplicati all'interno delle due entità figlie.

Il modello risulterebbe quindi composto da due entità distinte: una per i venditori e una per i meccanici.

Vantaggi:

- separazione più esplicita tra venditori e meccanici;
- possibilità di applicare vincoli specifici direttamente sulle singole entità;
- maggiore chiarezza nel caso in cui venditori e meccanici avessero molti attributi differenti.

Svantaggi:

- duplicazione degli attributi comuni nelle due entità figlie;
- maggiore complessità nella gestione dei dipendenti;
- necessità di interrogare due entità diverse per ottenere l'elenco completo del personale;
- minore flessibilità nel caso di introduzione di nuovi ruoli;
- possibile aumento della complessità nelle relazioni comuni ai dipendenti.

Questa soluzione sarebbe più adatta se venditori e meccanici avessero molti attributi specifici differenti. Nel caso di AutoGarage, invece, le due figure si distinguono principalmente per le operazioni che possono svolgere e non per un insieme significativo di attributi propri.

3.2.3 Scelta adottata

Nel progetto AutoGarage è stata adottata la strategia di **accorpamento al padre**. Le entità Venditore e Meccanico sono state eliminate come entità autonome e il loro significato è stato rappresentato tramite l'attributo **ruolo** dell'entità Dipendente.

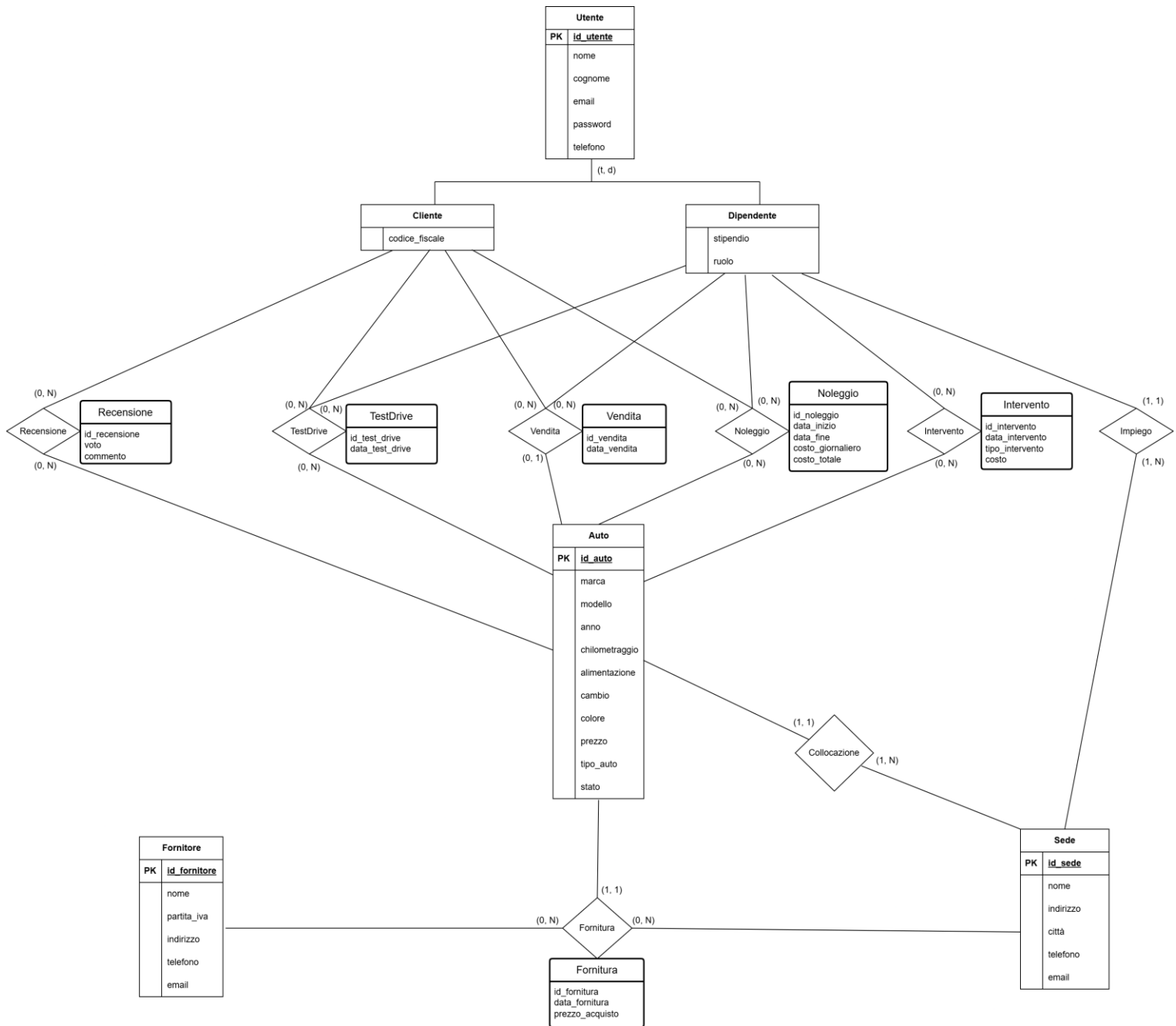
Questa scelta consente di semplificare il modello, ridurre il numero di entità e rendere più immediata la gestione del personale della concessionaria. Poiché venditori e meccanici non presentano attributi specifici propri nel modello concettuale, la distinzione tra le due categorie può essere gestita efficacemente tramite il ruolo.

Nel modello ristrutturato, i vincoli semantici vengono mantenuti stabilendo che:

- solo i dipendenti con ruolo **venditore** possono gestire test drive, vendite e noleggi;
- solo i dipendenti con ruolo **meccanico** possono effettuare interventi sui veicoli.

In questo modo il modello mantiene la distinzione funzionale tra le due tipologie di dipendenti, evitando però la complessità derivante dalla presenza di entità separate.

3.3 Modello E-R Intermedio



In questo nuovo modello la distinzione tra le due categorie di dipendenti non viene più rappresentata mediante entità separate, ma attraverso il valore assunto dall'attributo ruolo, che può identificare un dipendente come venditore oppure come meccanico. Le relazioni precedentemente associate alle due sottoclassi vengono quindi collegate direttamente all'entità Dipendente.

Questa soluzione consente di ridurre il numero complessivo di entità presenti nel modello e di semplificare la futura traduzione nello schema relazionale. Inoltre, poiché Venditore e Meccanico non possiedono attributi specifici propri ma si differenziano esclusivamente per le operazioni svolte all'interno della concessionaria, l'utilizzo dell'attributo ruolo risulta una rappresentazione adeguata e più efficiente.

Il modello intermedio mantiene tuttavia la generalizzazione tra **Utente**, **Cliente** e **Dipendente**, poiché queste entità presentano attributi distintivi significativi. In particolare, il Cliente possiede il codice fiscale mentre il Dipendente possiede informazioni relative all'attività lavorativa, come lo stipendio e il ruolo ricoperto all'interno della concessionaria. Per tale motivo si è ritenuto opportuno mantenere questa seconda gerarchia e procedere a una successiva valutazione delle possibili strategie di ristrutturazione.

3.4 Risoluzione della generalizzazione Utente – Cliente/Dipendente

3.4.1 Scenario 1 – Accorpamento al padre

La prima soluzione consiste nell'accorpare le entità figlie **Cliente** e **Dipendente** all'interno dell'entità padre **Utente**. In questo caso le entità Cliente e Dipendente vengono eliminate e tutti gli attributi vengono inseriti nell'entità Utente.

Per distinguere le diverse tipologie di utenti viene introdotto un attributo discriminante, ad esempio **ruolo**, che può assumere i valori "cliente", "venditore" e "meccanico". In questo modo la distinzione tra cliente e dipendente non viene più rappresentata tramite entità separate, ma tramite il valore del ruolo.

Vantaggi:

- riduzione del numero di entità e di tabelle nel modello logico;
- gestione centralizzata di tutti gli utenti del sistema;
- eliminazione della necessità di effettuare join tra Utente, Cliente e Dipendente;

- maggiore semplicità nelle operazioni di login, registrazione e autenticazione;
- maggiore flessibilità in caso di introduzione futura di nuovi ruoli;
- struttura più semplice da gestire lato applicativo.

Svantaggi:

- presenza di attributi non sempre valorizzati, come codice fiscale per i dipendenti o stipendio per i clienti;
- necessità di introdurre vincoli condizionali basati sul ruolo;
- minore separazione concettuale tra clienti e dipendenti;
- alcuni controlli semantici devono essere gestiti tramite vincoli o logica applicativa.

Questa soluzione è particolarmente adatta quando le diverse tipologie di utenti condividono molti attributi comuni e vengono gestite nello stesso sistema di autenticazione.

3.4.2 Scenario 2 – Accorpamento alle figlie

La seconda soluzione consiste nell'eliminare l'entità padre **Utente** e mantenere soltanto le entità figlie **Cliente** e **Dipendente**. In questo caso gli attributi comuni dell'utente, come nome, cognome, email, password e telefono, verrebbero duplicati nelle due entità figlie.

Il modello risulterebbe quindi composto da due entità principali separate: una per i clienti e una per i dipendenti.

Vantaggi:

- maggiore separazione tra clienti e dipendenti;
- tabelle più specifiche rispetto al ruolo dell'utente;
- possibilità di applicare direttamente vincoli diversi alle due categorie;
- assenza di campi nulli dovuti ad attributi non pertinenti.

Svantaggi:

- duplicazione degli attributi comuni;
- maggiore complessità nella gestione dell'autenticazione;
- necessità di controllare l'unicità dell'email su più entità;

- interrogazioni più complesse quando si vuole ottenere l'elenco completo degli utenti;
- minore flessibilità in caso di aggiunta di nuovi ruoli;
- maggiore complessità nella gestione delle relazioni comuni.

Questa soluzione sarebbe più adatta se clienti e dipendenti fossero soggetti completamente separati, con pochi attributi condivisi e processi applicativi indipendenti. Nel caso di AutoGarage, invece, tutti gli utenti accedono allo stesso sistema tramite credenziali e condividono molte informazioni anagrafiche comuni.

3.4.3 Scelta adottata

Nel progetto AutoGarage è stata adottata la strategia di **accorpamento al padre**. Le entità Cliente e Dipendente sono state eliminate come entità autonome e i loro attributi specifici sono stati inseriti all'interno dell'entità **Utente**.

La distinzione tra le diverse tipologie di utenti viene gestita tramite l'attributo **ruolo**, che consente di identificare se un utente è un cliente, un venditore o un meccanico.

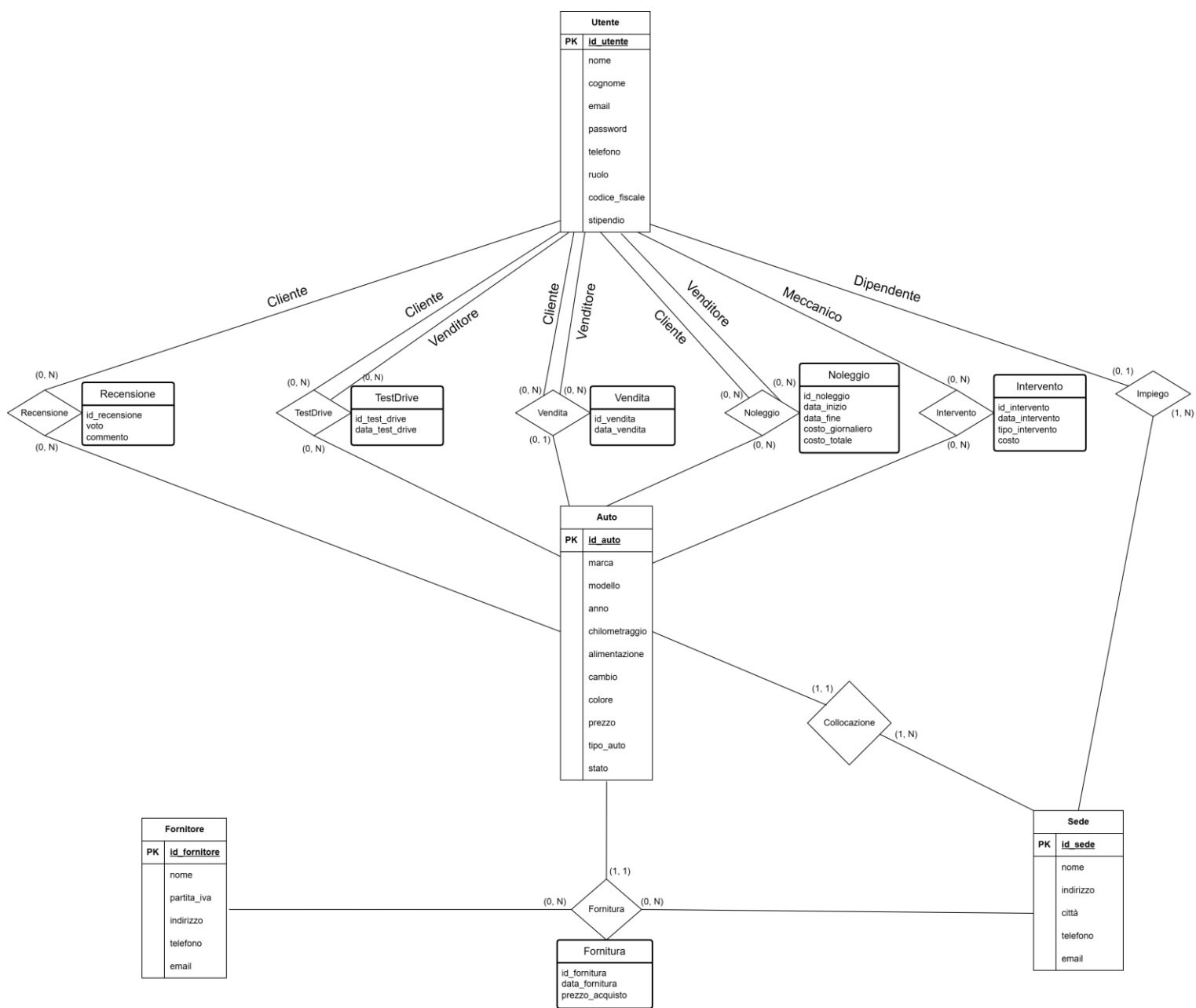
Questa scelta permette di semplificare il modello e di rendere più immediata la gestione dell'autenticazione, poiché tutti gli utenti sono memorizzati in un'unica entità. Inoltre, consente di evitare ridondanza sugli attributi comuni e di rendere più semplice l'accesso alle informazioni anagrafiche.

Nel modello finale vengono quindi mantenuti i vincoli semantici principali:

- gli utenti con ruolo **cliente** possono prenotare test drive, effettuare noleggi, acquistare auto e consultare recensioni;
- gli utenti con ruolo **venditore** possono gestire test drive, vendite e noleggi;
- gli utenti con ruolo **meccanico** possono gestire gli interventi sui veicoli.

In questo modo il modello mantiene la distinzione funzionale tra le diverse categorie di utenti, ma con una struttura più semplice e più adatta alla successiva implementazione relazionale.

3.5 Modello E-R Finale



A seguito della ristrutturazione delle generalizzazioni presenti nei modelli precedenti, è stato ottenuto il modello E-R finale del sistema AutoGarage.

Nel modello finale le entità **Cliente**, **Dipendente**, **Venditore** e **Meccanico** non sono più rappresentate come entità autonome. Tutte le informazioni relative agli utenti vengono infatti accorpate all'interno dell'unica entità **Utente**, che contiene sia gli attributi comuni sia gli attributi specifici delle diverse categorie di utenti.

L'entità **Utente** contiene quindi gli attributi:

id_utente, nome, cognome, email, password, telefono, ruolo, codice_fiscale, stipendio

L'attributo **ruolo** consente di distinguere le diverse tipologie di utenti previste dal sistema:

cliente, venditore, meccanico

Gli attributi **codice_fiscale** e **stipendio** assumono significato in base al ruolo dell'utente. In particolare, il codice fiscale è rilevante per gli utenti con ruolo cliente, mentre lo stipendio è rilevante per gli utenti con ruolo venditore o meccanico.

Le relazioni precedentemente collegate alle entità specializzate vengono ora collegate direttamente all'entità **Utente**, mantenendo però il significato semantico originario. Per questo motivo, nel modello finale vengono indicati i ruoli con cui l'utente partecipa alle diverse relazioni.

Questa soluzione consente di semplificare notevolmente il modello concettuale, riducendo il numero di entità e rendendo più diretta la futura traduzione nel modello logico relazionale. La gestione dei diversi profili utente viene affidata all'attributo ruolo, mentre i vincoli semantici vengono mantenuti attraverso opportune regole applicative e controlli sul database.

Nel modello finale restano inoltre presenti le entità principali del dominio:

- **Auto**, che rappresenta i veicoli gestiti dalla concessionaria;
- **Sede**, che rappresenta le diverse sedi operative;
- **Fornitore**, che rappresenta i fornitori dei veicoli;
- **Recensione**, **TestDrive**, **Vendita**, **Noleggio**, **Intervento** e **Fornitura**, che rappresentano le principali attività svolte nel sistema.

La scelta del modello finale permette quindi di ottenere una struttura più compatta e più facilmente traducibile in schema relazionale, mantenendo allo stesso tempo la distinzione funzionale tra clienti, venditori e meccanici.

3.6 Analisi delle cardinalità

Di seguito vengono analizzate le principali cardinalità presenti nel modello E-R finale.

Collocazione (Auto – Sede)

Cardinalità: (1,1) – (1,N)

Ogni automobile deve essere collocata obbligatoriamente in una sola sede della concessionaria, mentre una sede può contenere una o più automobili. Questa scelta permette di conoscere sempre la posizione di ogni veicolo presente nel catalogo.

Impiego (Utente – Sede)

Cardinalità: (0,1) – (1,N)

Ogni dipendente può essere assegnato ad una sola sede, mentre una sede può impiegare più dipendenti. In questo modo è possibile rappresentare l'organizzazione del personale all'interno delle diverse filiali.

Recensione (Cliente – Auto)

Cardinalità: (0,N) – (0,N)

Un cliente può scrivere nessuna, una o più recensioni. Allo stesso tempo, un'automobile può ricevere recensioni da più clienti. La partecipazione è opzionale per entrambe le entità.

TestDrive (Cliente – Venditore – Auto)

Cardinalità: (0,N)

Un cliente può prenotare più test drive nel corso del tempo, un venditore può gestire più prenotazioni e una stessa automobile può essere utilizzata per più prove su strada.

Vendita (Cliente – Venditore – Auto)

Cardinalità: (0,N) per Cliente e Venditore, (0,1) per Auto.

Un cliente può acquistare più automobili e un venditore può effettuare più vendite. Un'automobile, invece, può essere venduta una sola volta, motivo per cui la cardinalità massima sul lato Auto è pari a 1.

Noleggio (Cliente – Venditore – Auto)

Cardinalità: (0,N)

Un cliente può effettuare più noleggi nel tempo, un venditore può gestire più contratti e una stessa automobile può essere noleggiata in periodi differenti.

Intervento (Meccanico – Auto)

Cardinalità: (0,N) – (0,N)

Un meccanico può effettuare più interventi e un'automobile può essere sottoposta a numerosi interventi durante il proprio ciclo di vita.

Fornitura (Fornitore – Auto – Sede)

Cardinalità: (0,N)

Un fornitore può effettuare numerose forniture, una sede può ricevere più automobili e ogni fornitura registra l'approvvigionamento di uno specifico veicolo destinato ad una determinata sede.

4 Progettazione Logica

La progettazione logica è stata ottenuta a partire dal modello E-R finale attraverso la traduzione delle entità e delle relazioni nello schema relazionale. Durante questa fase sono state definite le chiavi primarie e le chiavi esterne necessarie a garantire l'integrità referenziale del database.

Lo schema logico risultante è il seguente:

- **Utente** (id_utente, nome, cognome, email, password, telefono, ruolo, codice_fiscale, stipendio)
- **Auto** (id_auto, marca, modello, anno, chilometraggio, alimentazione, cambio, colore, prezzo, tipo_auto, stato, id_sede: Sede)
- **Fornitore** (id_fornitore, nome, partita_iva, indirizzo, telefono, email)
- **Sede** (id_sede, nome, indirizzo, città, telefono, email)
- **Recensione** (id_recensione, voto, commento, id_cliente: Utente, id_auto: Auto)
- **TestDrive** (id_test_drive, data_test_drive, id_cliente: Utente, id_auto: Auto, id_venditore: Utente)
- **Vendita** (id_vendita, data_vendita, id_cliente: Utente, id_auto: Auto, id_venditore: Utente)
- **Noleggio** (id_noleggio, data_inizio, data_fine, costo_giornaliero, costo_totale, id_cliente: Utente, id_auto: Auto, id_venditore: Utente)
- **Intervento** (id_intervento, data_intervento, tipo_intervento, costo, id_auto: Auto, id_meccanico: Utente)
- **Fornitura** (id_fornitura, data_fornitura, prezzo_acquisto, id_fornitore: Fornitore, id_auto: Auto, id_sede: Sede)
- **Impiego** (id_dipendente: Utente, id_sede: Sede)

4.1 Struttura tabelle

4.1.1 Utente

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco dell'utente	INT	-	PK, AUTO_INCREMENT
nome	Nome dell'utente	VARCHAR	100	NOT NULL
cognome	Cognome dell'utente	VARCHAR	100	NOT NULL
email	Email utilizzata per l'accesso	VARCHAR	254	UNIQUE, NOT NULL
password	Password dell'utente	VARCHAR	200	NOT NULL
telefono	Numero di telefono dell'utente	VARCHAR	20	NOT NULL
ruolo	Ruolo dell'utente nel sistema	VARCHAR	20	NOT NULL
codice_fiscale	Codice fiscale del cliente	VARCHAR	16	UNIQUE, NULLABLE
stipendio	Stipendio del dipendente	DECIMAL	10,2	NULLABLE

4.1.2 Auto

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco dell'auto	INT	-	PK, AUTO_INCREMENT
marca	Marca dell'automobile	VARCHAR	100	NOT NULL
modello	Modello dell'automobile	VARCHAR	100	NOT NULL
anno	Anno di immatricolazione	INT	-	NOT NULL
chilometraggio	Chilometraggio dell'automobile	INT	-	NOT NULL
alimentazione	Tipo di alimentazione	VARCHAR	50	NOT NULL
cambio	Tipo di cambio	VARCHAR	50	NOT NULL
colore	Colore dell'automobile	VARCHAR	50	NOT NULL
prezzo	Prezzo dell'automobile	DECIMAL	10,2	NOT NULL
tipo_auto	Tipologia dell'auto	VARCHAR	20	NOT NULL
stato	Stato dell'automobile	VARCHAR	20	NOT NULL
sede_id	Sede in cui si trova l'auto	INT	-	FK → Sede(id), NOT NULL

4.1.3 Sede

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco della sede	INT	-	PK, AUTO_INCREMENT
nome	Nome della sede	VARCHAR	100	UNIQUE, NOT NULL
indirizzo	Indirizzo della sede	VARCHAR	200	NOT NULL
citta	Città in cui si trova la sede	VARCHAR	100	NOT NULL
telefono	Numero di telefono della sede	VARCHAR	20	NOT NULL
email	Email della sede	VARCHAR	254	UNIQUE, NOT NULL

4.1.4 Fornitore

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco del fornitore	INT	-	PK, AUTO_INCREMENT
nome	Nome del fornitore	VARCHAR	100	NOT NULL
partita_iva	Partita IVA del fornitore	VARCHAR	20	UNIQUE, NOT NULL
indirizzo	Indirizzo del fornitore	VARCHAR	200	NOT NULL
telefono	Numero di telefono del fornitore	VARCHAR	20	NOT NULL
email	Email del fornitore	VARCHAR	254	UNIQUE, NOT NULL

4.1.5 Recensione

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco della recensione	INT	-	PK, AUTO_INCREMENT
voto	Valutazione numerica assegnata all'auto	INT	-	NOT NULL
commento	Testo della recensione	LONGTEXT	-	NOT NULL
auto_id	Auto recensita	INT	-	FK → Auto(id), NOT NULL
cliente_id	Cliente che ha scritto la recensione	INT	-	FK → Utente(id), NOT NULL

4.1.6 TestDrive

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco del test drive	INT	-	PK, AUTO_INCREMENT
data_test_drive	Data prevista per il test drive	DATE	-	NOT NULL
auto_id	Auto oggetto del test drive	INT	-	FK → Auto(id), NOT NULL
cliente_id	Cliente che prenota il test drive	INT	-	FK → Utente(id), NOT NULL

venditore_id	Venditore che gestisce il test drive	INT	-	FK → Utente(id), NOT NULL
--------------	--------------------------------------	-----	---	---------------------------

4.1.7 Vendita

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco della vendita	INT	-	PK, AUTO_INCREMENT
data_vendita	Data in cui viene effettuata la vendita	DATE	-	NOT NULL
auto_id	Auto venduta	INT	-	FK → Auto(id), UNIQUE, NOT NULL
cliente_id	Cliente che acquista l'auto	INT	-	FK → Utente(id), NOT NULL
venditore_id	Venditore che gestisce la vendita	INT	-	FK → Utente(id), NOT NULL

4.1.8 Noleggio

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco del noleggio	INT	-	PK, AUTO_INCREMENT
data_inizio	Data di inizio del noleggio	DATE	-	NOT NULL
data_fine	Data di fine del noleggio	DATE	-	NOT NULL
costo_giornaliero	Costo giornaliero del noleggio	DECIMAL	10,2	NOT NULL

costo_totale	Costo complessivo del noleggio	DECIMAL	10,2	NOT NULL
auto_id	Auto noleggiata	INT	-	FK → Auto(id), NOT NULL
cliente_id	Cliente che effettua il noleggio	INT	-	FK → Utente(id), NOT NULL
venditore_id	Venditore che gestisce il noleggio	INT	-	FK → Utente(id), NOT NULL

4.1.9 Intervento

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco dell'intervento	INT	-	PK, AUTO_INCREMENT
data_intervento	Data dell'intervento	DATE	-	NOT NULL
tipo_intervento	Tipologia dell'intervento effettuato	VARCHAR	100	NOT NULL
costo	Costo dell'intervento	DECIMAL	10,2	NOT NULL
auto_id	Auto sottoposta all'intervento	INT	-	FK → Auto(id), NOT NULL

meccanico_id	Meccanico che effettua l'intervento	INT	-	FK → Utente(id), NOT NULL
--------------	-------------------------------------	-----	---	---------------------------

4.1.10 Fornitura

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco della fornitura	INT	-	PK, AUTO_INCREMENT
data_fornitura	Data della fornitura	DATE	-	NOT NULL
prezzo_acquisto	Prezzo di acquisto dell'auto	DECIMAL	10,2	NOT NULL
auto_id	Auto fornita	INT	-	FK → Auto(id), UNIQUE, NOT NULL
fornitore_id	Fornitore dell'auto	INT	-	FK → Fornitore(id), NOT NULL
sede_id	Sede destinataria della fornitura	INT	-	FK → Sede(id), NOT NULL

4.1.11 Impiego

Nome Campo	Descrizione	Tipo Dati	Lunghezza	Vincoli
id	Identificativo univoco dell'impiego	INT	-	PK, AUTO_INCREMENT
sede_id	Sede presso cui lavora il dipendente	INT	-	FK → Sede(id), NOT NULL

dipendente_id	Dipendente assegnato alla sede	INT	-	FK → Utente(id), NOT NULL
---------------	--------------------------------	-----	---	---------------------------

4.2 Vincoli interrelazionali

- Gli utenti con ruolo **cliente** possono partecipare alle relazioni **Recensione**, **TestDrive**, **Vendita** e **Noleggio** nel ruolo di cliente.
- Gli utenti con ruolo **venditore** possono gestire **Vendita**, **TestDrive** e **Noleggio**.
- Gli utenti con ruolo **meccanico** possono eseguire **Intervento**.
- Alla relazione **Impiego** possono partecipare solo utenti con ruolo **venditore** o **meccanico**.
- Ogni utente con ruolo **venditore** o **meccanico** deve essere assegnato a una sola sede.
- Ogni auto deve provenire da una e una sola **fornitura**.
- Ogni fornitura deve essere associata a un solo **fornitore**, a una sola **auto** e a una sola **sede**.
- Una vendita può riguardare solo un'auto **disponibile** e non ancora venduta.
- Dopo la vendita, lo stato dell'auto deve diventare **venduta**.
- Un'auto venduta non può essere **noleggiata**.
- Un'auto venduta non può essere prenotata per un **test drive**.
- Un test drive può essere effettuato solo su un'auto **disponibile**.

- Un noleggio può riguardare solo un'auto disponibile nel periodo richiesto.
- La stessa auto non può essere associata a due noleggi con periodi sovrapposti.
- Durante il periodo di noleggio, l'auto non può essere venduta.
- Un'auto in manutenzione non può essere noleggiata o prenotata per un test drive.
- Un cliente può lasciare una recensione su un'auto solo se l'ha acquistata, noleggiata o provata tramite test drive.
- Un **cliente** non può noleggiare un'auto se nello stesso periodo l'auto è già noleggiata da un altro cliente.
- Un **test drive** deve essere gestito da un venditore appartenente a una sede coerente con la sede in cui si trova l'auto.
- Una **vendita** deve essere gestita da un venditore appartenente alla sede in cui si trova l'auto.
- Un **noleggio** deve essere gestito da un venditore appartenente alla sede in cui si trova l'auto.
- Una **fornitura** determina l'ingresso dell'auto nel catalogo della sede indicata.
- Un'auto può essere recensita da un cliente solo dopo una relazione effettiva con essa: vendita, noleggio o test drive.

4.3 Trigger implementati

Per garantire il rispetto di alcuni vincoli semantici non esprimibili tramite semplici chiavi esterne, nel database MySQL sono stati implementati diversi trigger. Tali trigger vengono eseguiti automaticamente prima o dopo l'inserimento di nuovi record nelle tabelle principali e, in caso di violazione, bloccano l'operazione tramite SIGNAL SQLSTATE '45000'.

4.3.1 Trigger before_insert_vendita

Il trigger before_insert_vendita viene eseguito prima dell'inserimento di una nuova vendita nella tabella garage_vendita. Il suo scopo è garantire che la vendita rispetti i vincoli semantici del sistema. In particolare, verifica che l'utente associato come cliente abbia ruolo cliente, che l'utente associato come venditore abbia ruolo venditore e che l'automobile coinvolta nella vendita sia ancora disponibile. Se uno di questi controlli non viene rispettato, l'inserimento viene bloccato tramite SIGNAL SQLSTATE '45000'.

```
IF ruolo_cliente <> 'cliente' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: solo un cliente può acquistare un auto';
END IF;

IF ruolo_venditore <> 'venditore' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: la vendita deve essere gestita da un venditore';
END IF;

IF stato_auto <> 'disponibile' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: si può vendere solo un auto disponibile';
END IF;
END$$
```

4.3.2 Trigger after_insert_vendita

Il trigger after_insert_vendita viene eseguito dopo l'inserimento di una nuova vendita nella tabella garage_vendita. Il suo compito è aggiornare automaticamente lo stato dell'automobile venduta, impostandolo a venduta. In questo modo, dopo la registrazione della vendita, il veicolo non risulta più disponibile per ulteriori vendite, noleggi o test drive.

```
UPDATE garage_auto
SET stato = 'venduta'
WHERE id = NEW.auto_id;
```

4.3.3 Trigger before_insert_noleggio

Il trigger before_insert_noleggio viene eseguito prima dell'inserimento di un nuovo noleggio nella tabella garage_noleggio. Il suo scopo è verificare che il noleggio rispetti i ruoli previsti dal sistema. In particolare, controlla che l'utente associato come cliente abbia ruolo cliente e

che l'utente incaricato della gestione del noleggio abbia ruolo venditore. Se uno dei due controlli non viene rispettato, l'operazione viene bloccata.

```
IF ruolo_cliente <> 'cliente' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: solo un cliente puo effettuare un noleggio';
END IF;

IF ruolo_venditore <> 'venditore' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: il noleggio deve essere gestito da un venditore';
END IF;
```

4.3.4 Trigger before_insert_testdrive

Il trigger before_insert_testdrive viene eseguito prima dell'inserimento di una nuova prenotazione nella tabella garage_testdrive. Il trigger verifica che il test drive venga richiesto da un utente con ruolo cliente e che venga gestito da un utente con ruolo venditore. In questo modo si impedisce che utenti con ruoli non corretti possano partecipare alla relazione con funzioni diverse da quelle previste dal modello.

```
IF ruolo_cliente <> 'cliente' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: solo un cliente puo effettuare un test drive';
END IF;

IF ruolo_venditore <> 'venditore' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: il test drive deve essere gestito da un venditore';
END IF;
```

4.3.5 Trigger before_insert_intervento

Il trigger before_insert_intervento viene eseguito prima dell'inserimento di un nuovo intervento nella tabella garage_intervento. Il suo scopo è verificare che l'utente associato all'intervento abbia ruolo meccanico. In questo modo il database impedisce che un intervento di manutenzione venga assegnato a un cliente o a un venditore, mantenendo la coerenza con i vincoli del modello E-R.

```
IF ruolo_mecanico <> 'meccanico' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: l intervento deve essere effettuato da un meccanico';
END IF;
```

4.3.6 Trigger before_insert_recensione

Il trigger before_insert_recensione viene eseguito prima dell'inserimento di una nuova recensione nella tabella garage_recensione. Il suo scopo è verificare che l'utente associato alla recensione abbia ruolo cliente. In questo modo il sistema impedisce che utenti con ruolo

venditore o meccanico possano inserire recensioni, mantenendo coerente il significato della relazione Recensione.

```
IF ruolo_cliente <> 'cliente' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: solo un cliente può scrivere una recensione';
END IF;
```

4.3.7 Trigger before_insert_impiego

Il trigger before_insert_impiego viene eseguito prima dell'inserimento di un nuovo record nella tabella garage_impiego. Il suo scopo è verificare che l'utente assegnato a una sede sia effettivamente un dipendente della concessionaria, quindi un utente con ruolo venditore oppure meccanico. In questo modo si impedisce che un cliente venga associato a una sede come membro del personale.

```
IF ruolo_dipendente NOT IN ('venditore', 'meccanico') THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Errore: solo venditori e meccanici possono essere impiegati in una sede';
END IF;
```

5 Implementazione del sistema informativo

L'applicazione AutoGarage è stata sviluppata utilizzando il framework Django. Il sistema utilizza MySQL come DBMS per la gestione dei dati e Bootstrap per la realizzazione dell'interfaccia grafica.

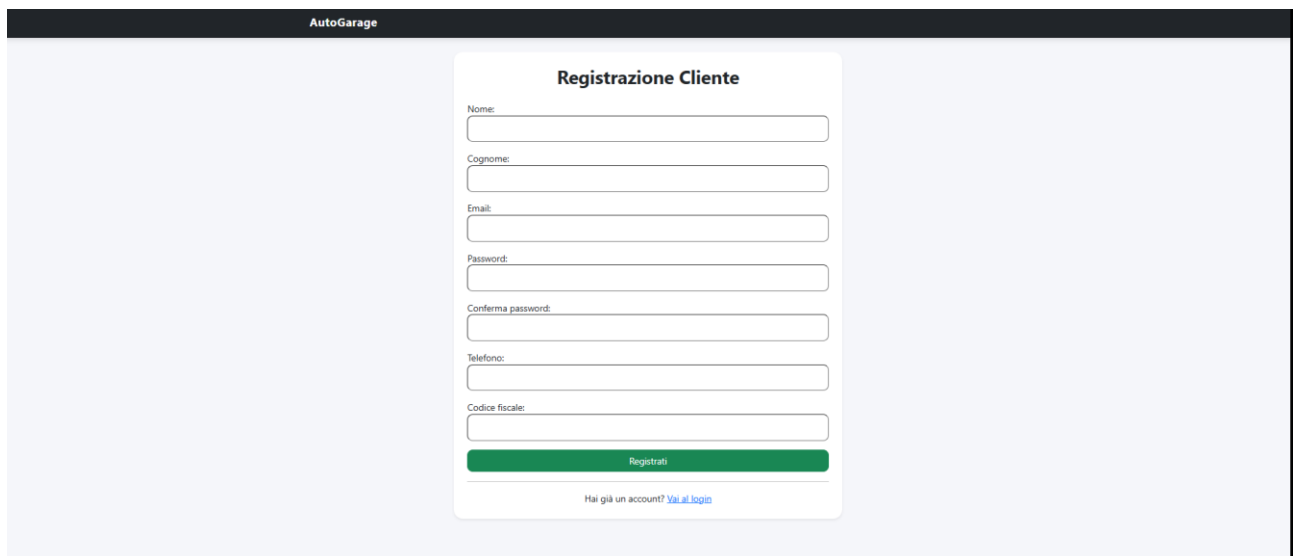
L'applicazione consente la gestione delle principali attività della concessionaria, permettendo agli utenti di consultare il catalogo delle auto, prenotare test drive, registrare vendite, gestire gli interventi di officina e inserire recensioni. L'accesso alle funzionalità è regolato da un sistema di autenticazione che distingue clienti, venditori e meccanici in base al ruolo associato all'account.

5.1 Login

Registrati'." data-bbox="115 366 932 613"/>

La schermata di login rappresenta il punto di accesso principale al sistema AutoGarage. Gli utenti registrati possono autenticarsi inserendo il proprio indirizzo e-mail e la password associata all'account. Il sistema verifica le credenziali inserite e, in caso di autenticazione corretta, reindirizza l'utente alle funzionalità disponibili in base al ruolo associato (cliente, venditore o meccanico). Le password vengono gestite in modo sicuro tramite i meccanismi di autenticazione forniti dal framework Django.

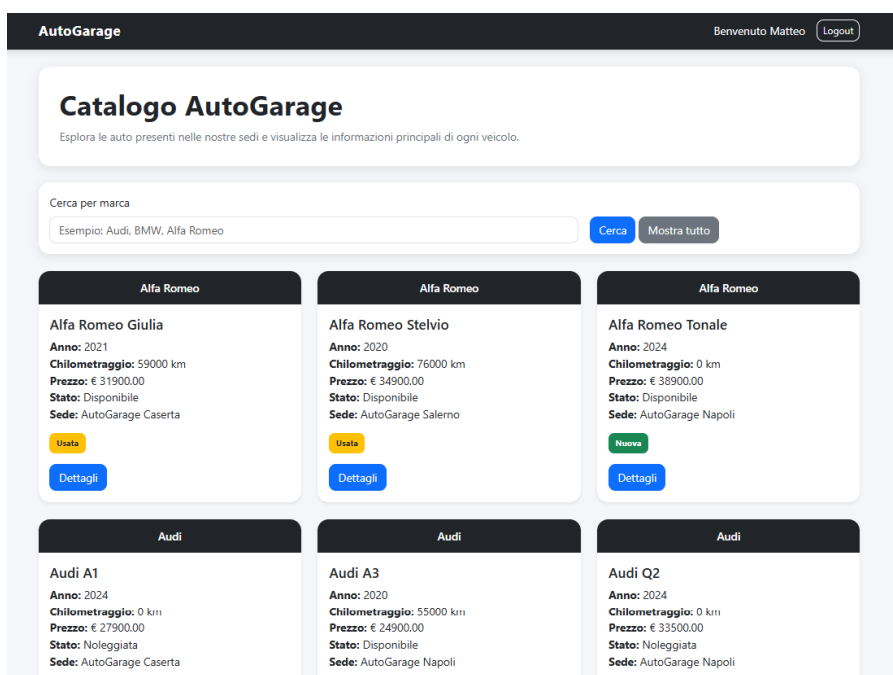
5.2 Registrazione



The screenshot shows the 'Registrazione Cliente' (Client Registration) form within the AutoGarage application. The form is titled 'Registrazione Cliente' and contains the following fields: Nome (Name), Cognome (Surname), Email, Password, Conferma password (Confirm password), Telefono (Phone), and Codice fiscale (Tax code). A green 'Registrati' (Register) button is located at the bottom of the form. Below the button, there is a link: 'Hai già un account? [Vai al login](#)'.

La schermata di registrazione consente a un nuovo cliente di creare un account all'interno del sistema AutoGarage. Durante la compilazione vengono richiesti i principali dati personali, come nome, cognome, e-mail, password e telefono. Una volta completata la registrazione, l'utente viene automaticamente registrato come cliente e può accedere alle funzionalità a lui dedicate, come la consultazione del catalogo delle auto, la prenotazione dei test drive e l'inserimento di recensioni. Gli account dei dipendenti, invece, vengono creati e gestiti direttamente dall'amministratore del sistema, che assegna le credenziali e il ruolo appropriato (venditore o meccanico).

5.3 Catalogo Auto



The screenshot displays the 'Catalogo AutoGarage' (AutoGarage Car Catalog) page. The header shows 'AutoGarage' and 'Benvenuto Matteo' with a 'Logout' button. The main heading is 'Catalogo AutoGarage' with a subtitle: 'Esplora le auto presenti nelle nostre sedi e visualizza le informazioni principali di ogni veicolo.' Below this is a search bar labeled 'Cerca per marca' with the example text 'Esempio: Audi, BMW, Alfa Romeo' and buttons for 'Cerca' and 'Mostra tutto'. The catalog is organized into a grid of car cards. The first row features three Alfa Romeo cars: Giulia, Stelvio, and Tonale. The second row features three Audi cars: A1, A3, and Q2. Each card displays the car's name, year, mileage, price, status, and location, along with a 'Usata' (Used) or 'Nuova' (New) badge and a 'Dettagli' (Details) button.

Alfa Romeo	Alfa Romeo	Alfa Romeo
Alfa Romeo Giulia Anno: 2021 Chilometraggio: 59000 km Prezzo: € 31900.00 Stato: Disponibile Sede: AutoGarage Caserta Usata Dettagli	Alfa Romeo Stelvio Anno: 2020 Chilometraggio: 76000 km Prezzo: € 34900.00 Stato: Disponibile Sede: AutoGarage Salerno Usata Dettagli	Alfa Romeo Tonale Anno: 2024 Chilometraggio: 0 km Prezzo: € 38900.00 Stato: Disponibile Sede: AutoGarage Napoli Nuova Dettagli

Audi	Audi	Audi
Audi A1 Anno: 2024 Chilometraggio: 0 km Prezzo: € 27900.00 Stato: Noleggiata Sede: AutoGarage Caserta Nuova Dettagli	Audi A3 Anno: 2020 Chilometraggio: 55000 km Prezzo: € 24900.00 Stato: Disponibile Sede: AutoGarage Napoli Usata Dettagli	Audi Q2 Anno: 2024 Chilometraggio: 0 km Prezzo: € 33500.00 Stato: Noleggiata Sede: AutoGarage Napoli Nuova Dettagli

La schermata del catalogo permette agli utenti di visualizzare le auto presenti nel sistema AutoGarage. Per ogni veicolo vengono mostrate le informazioni principali, come marca, modello, anno, chilometraggio e prezzo. Da questa sezione l'utente può consultare rapidamente le auto disponibili e accedere alla scheda dettagliata del veicolo selezionato.

5.4 Dettaglio Auto

AutoGarage Benvenuto Matteo Logout

Alfa Romeo Giulia

Anno: 2021
Chilometraggio: 59000 km
Alimentazione: diesel
Cambio: automatico
Colore: rosso
Prezzo: € 31900.00
Tipo: Usata
Stato: Disponibile
Sede: AutoGarage Caserta

[Torna al catalogo](#) [Prenota test drive](#)

Recensioni

Nessuna recensione presente per questa auto.

La schermata di dettaglio consente di visualizzare le informazioni complete relative a una singola auto selezionata dal catalogo. Oltre ai dati principali del veicolo, come marca, modello, anno, chilometraggio e prezzo, vengono mostrati anche elementi utili alla scelta dell'utente, come alimentazione, cambio, colore, tipologia e stato dell'auto. Da questa pagina l'utente può consultare in modo più approfondito il veicolo di interesse.

5.5 Prenotazione Test Drive

The screenshot shows the 'Prenota test drive' form. At the top, the header 'AutoGarage' is on the left, and 'Benvenuto Matteo' with a 'Logout' button is on the right. The main heading is 'Prenota test drive' with a subtext 'Scegli una data per provare il veicolo selezionato.' Below this, a card displays vehicle details: 'Alfa Romeo Giulia', 'Anno: 2021', 'Prezzo: € 31900.00', and 'Sede: AutoGarage Caserta'. A date selection field is labeled 'Data test drive:' with a placeholder 'dd/mm/yyyy' and a calendar icon. At the bottom, there are two buttons: 'Conferma prenotazione' (green) and 'Annulla' (grey).

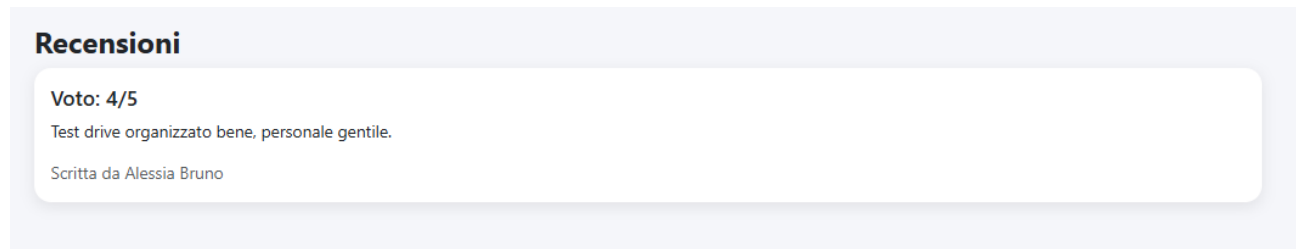
La funzionalità di prenotazione dei test drive consente ai clienti di richiedere una prova su strada per un veicolo presente nel catalogo. Ogni prenotazione viene associata a un cliente, a un'auto e a un dipendente con ruolo di venditore. Il sistema verifica inoltre il rispetto dei vincoli definiti nel database, impedendo la prenotazione di veicoli non disponibili, venduti o in manutenzione.

5.6 Ricerca auto per marca

The screenshot shows the 'Catalogo AutoGarage' search interface. The header is identical to the previous page. The main heading is 'Catalogo AutoGarage' with a subtext 'Esplora le auto presenti nelle nostre sedi e visualizza le informazioni principali di ogni veicolo.' Below this, there is a search section titled 'Cerca per marca' with a text input field containing the placeholder 'Esempio: Audi, BMW, Alfa Romeo'. To the right of the input field are two buttons: 'Cerca' (blue) and 'Mostra tutto' (grey).

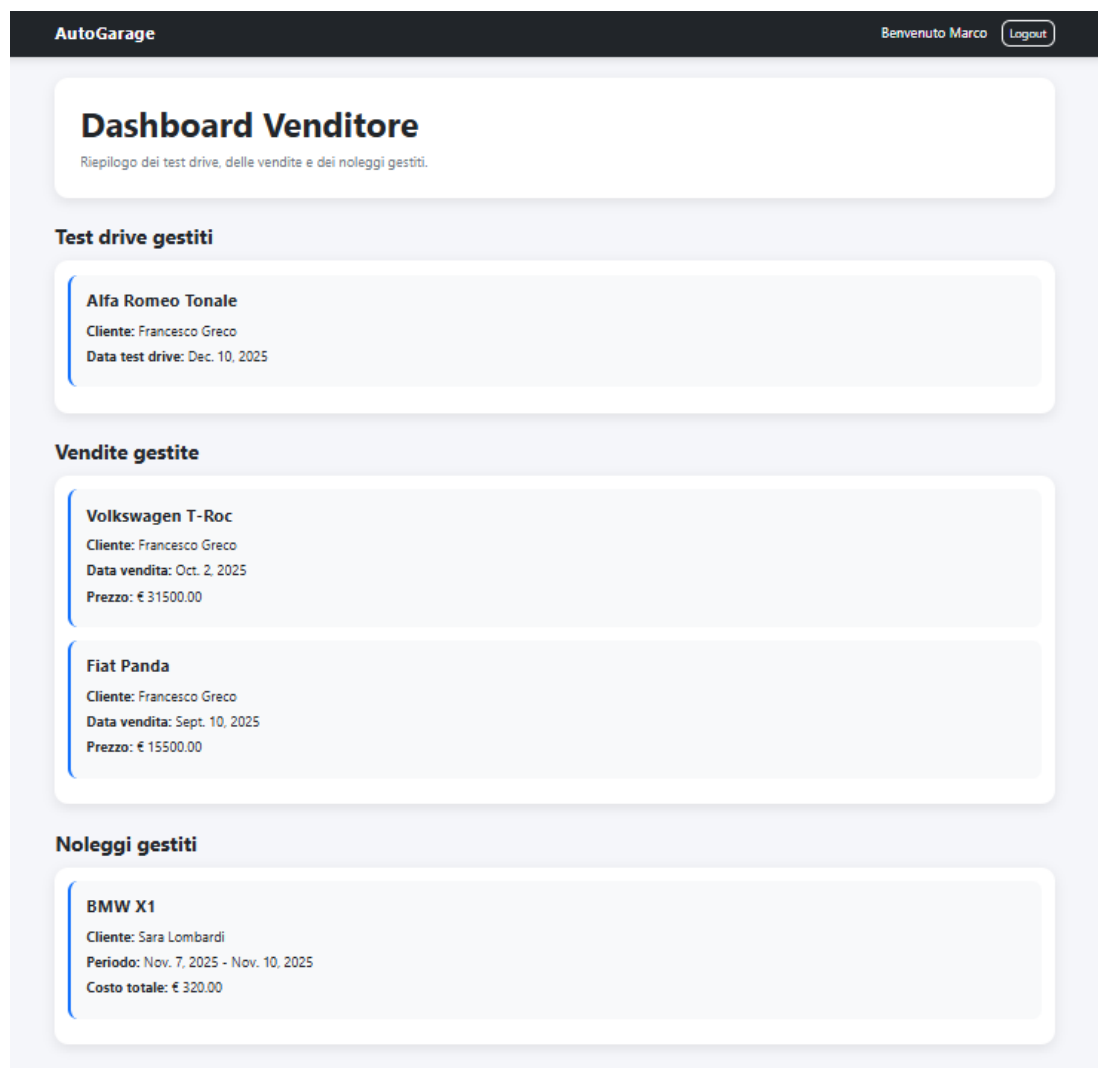
La schermata del catalogo include una funzionalità di ricerca che consente agli utenti di filtrare rapidamente i veicoli in base alla marca. L'utente può inserire il nome della marca desiderata nell'apposito campo di ricerca e visualizzare solamente le auto corrispondenti ai criteri inseriti. È inoltre disponibile il pulsante "Mostra tutto", che permette di ripristinare la visualizzazione completa del catalogo. Questa funzionalità facilita la consultazione dei veicoli e rende più rapida l'individuazione delle auto di interesse.

5.7 Visualizzazione recensioni



La schermata delle recensioni permette di visualizzare i commenti e le valutazioni lasciate dai clienti sulle auto presenti nel sistema. Ogni recensione è associata a un veicolo e contiene un voto numerico e un commento testuale. Questa funzionalità consente agli utenti di consultare le esperienze degli altri clienti e ottenere informazioni aggiuntive prima di interessarsi a un'auto.

5.8 Dashboard venditore



La dashboard del venditore raccoglie le informazioni e le funzionalità riservate agli utenti con ruolo venditore. Da questa sezione il venditore può consultare le attività di propria competenza, come la gestione dei test drive, delle vendite, e dei noleggi. L'accesso a questa pagina è regolato dal sistema di autenticazione e autorizzazione basato sui ruoli.

5.9 Dashboard meccanico

The screenshot shows the 'Dashboard Meccanico' interface. At the top, there is a dark header with 'AutoGarage' on the left and 'Benvenuto Antonio' with a 'Logout' button on the right. Below the header, the main content area has a white background. It starts with the title 'Dashboard Meccanico' in bold, followed by a subtitle 'Elenco degli interventi assegnati e delle attività di officina.' Below this, there is a section titled 'Interventi assegnati'. Under this section, there is a card for a 'Fiat 500' with the following details: 'Tipo intervento: Tagliando', 'Data: Oct. 20, 2025', and 'Costo: € 180.00'.

AutoGarage Benvenuto Antonio [Logout](#)

Dashboard Meccanico

Elenco degli interventi assegnati e delle attività di officina.

Interventi assegnati

Fiat 500
Tipo intervento: Tagliando
Data: Oct. 20, 2025
Costo: € 180.00

La dashboard del meccanico è dedicata agli utenti con ruolo meccanico e permette di accedere alle funzionalità relative agli interventi di officina. In questa sezione il meccanico può visualizzare le attività assegnate e consultare le informazioni utili alla gestione degli interventi sui veicoli. Anche questa schermata è protetta dal sistema di controllo degli accessi basato sui ruoli.

6 Gestione delle sessioni

Nel progetto AutoGarage è stato utilizzato il sistema di sessioni fornito da Django per mantenere lo stato dell'utente durante la navigazione all'interno dell'applicazione. Poiché il protocollo HTTP è stateless, ogni richiesta inviata al server è indipendente dalle precedenti; per questo motivo è necessario salvare alcune informazioni temporanee che permettano al sistema di riconoscere l'utente autenticato tra una pagina e l'altra.

La gestione delle sessioni è abilitata nel progetto tramite l'applicazione `django.contrib.sessions`, inserita all'interno di `INSTALLED_APPS`.

```
'django.contrib.sessions',
```

Inoltre, nel file di configurazione è presente il middleware `django.contrib.sessions.middleware.SessionMiddleware`, che permette a Django di associare una sessione alle richieste HTTP effettuate dall'utente.

```
'django.contrib.sessions.middleware.SessionMiddleware',
```

Quando un utente effettua correttamente il login, il sistema salva nella sessione alcune informazioni fondamentali, come l'identificativo dell'utente, il nome e il ruolo. In particolare, vengono memorizzati `utente_id`, `utente_nome` e `utente_ruolo`. Questi dati consentono al sistema di riconoscere l'utente nelle pagine successive e di distinguere le funzionalità disponibili in base al ruolo, cioè cliente, venditore o meccanico.

```
request.session["utente_id"] = utente.id  
request.session["utente_nome"] = utente.nome  
request.session["utente_ruolo"] = utente.ruolo
```

Al momento del logout, la sessione viene eliminata tramite il metodo `request.session.flush()`. In questo modo tutte le informazioni salvate nella sessione vengono rimosse e l'utente, per accedere nuovamente alle aree riservate, dovrà effettuare un nuovo login.

```
request.session.flush()
```

L'utilizzo delle sessioni risulta quindi fondamentale per il corretto funzionamento del sistema di autenticazione, per la gestione dei ruoli e per la protezione delle funzionalità riservate agli utenti autenticati.

7 Attacchi e contromisure

7.1 SQL Injection

La SQL Injection è una tecnica di attacco che consiste nell'inserimento di istruzioni SQL malevole all'interno dei campi di input di un'applicazione web, con l'obiettivo di modificare il comportamento delle query eseguite sul database.

The image shows two side-by-side screenshots of a web application's login page. Both pages have a title 'Login' at the top. The left page shows a successful login attempt with the email 'matteo.serra@email.it' and a masked password. The right page shows a failed login attempt with the same email and password, displaying a red error message: 'Email o password non corrette. Tentativi rimasti: 1.' Below the input fields is a blue button labeled 'Accedi'. At the bottom of both pages is a link: 'Non hai un account? [Registrati](#)'.

Per verificare la resistenza del sistema AutoGarage a questo tipo di attacco, è stato simulato l'inserimento della stringa malevola ' OR '1'='1 nei campi di autenticazione. Il tentativo non ha prodotto alcun accesso non autorizzato e il sistema ha correttamente negato il login.

```
email = form.cleaned_data["email"]
password = form.cleaned_data["password"]
```

```
utente = authenticate(request, username=email, password=password)

if utente is None:
    raise Utente.DoesNotExist
```

Le credenziali inserite dall'utente vengono preliminarmente validate tramite i form Django e successivamente verificate mediante la funzione `authenticate()`. Questo approccio evita la costruzione manuale di query SQL e consente al framework di gestire in modo sicuro il processo di autenticazione.

```
def save(self, commit=True): 4 usages (3 dynamic) Stefano Vanzini *
    utente = super().save(commit=False)
    utente.password = make_password(self.cleaned_data["password"])
    utente.ruolo = "cliente"

    if commit:
        utente.save()

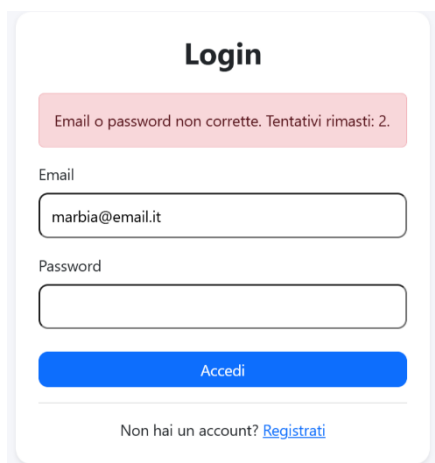
    return utente
```

Durante la registrazione, le password vengono sottoposte a hashing tramite la funzione `make_password()` prima di essere salvate nel database. In questo modo le credenziali non vengono memorizzate in chiaro e risultano maggiormente protette contro accessi non autorizzati.

7.2 Attacco a forza bruta

L'attacco a forza bruta consiste nel tentare ripetutamente l'accesso a un sistema provando numerose combinazioni di credenziali fino a individuare quella corretta. Questo tipo di attacco viene spesso automatizzato mediante software che eseguono migliaia di tentativi consecutivi in tempi molto ridotti.

Nel progetto AutoGarage è stata implementata una contromisura che limita il numero di tentativi di autenticazione consentiti. Dopo tre tentativi consecutivi falliti, l'account viene temporaneamente bloccato per due minuti, impedendo ulteriori accessi.



Login

Email o password non corrette. Tentativi rimasti: 2.

Email
marbia@email.it

Password

Accedi

Non hai un account? [Registrati](#)

Al primo tentativo di accesso con credenziali errate, il sistema registra l'errore e informa l'utente del numero di tentativi ancora disponibili prima del blocco temporaneo dell'account.

The image shows a login form titled "Login". At the top, a pink message box says "Email o password non corrette. Tentativi rimasti: 1." Below this, there are two input fields: "Email" with the value "marbia@email.it" and "Password" which is empty. A blue button labeled "Accedi" is below the password field. At the bottom, there is a link "Registrati" preceded by the text "Non hai un account?".

Il sistema continua a monitorare i tentativi di autenticazione falliti mantenendo un contatore associato all'utente che sta effettuando il login.

The image shows the same login form as before, but the pink message box now says "Hai effettuato 3 tentativi errati. Account bloccato per 2 minuti." The email field still contains "marbia@email.it" and the password field is empty. The "Accedi" button and the "Registrati" link remain the same.

Dopo tre tentativi consecutivi non riusciti, il sistema attiva automaticamente un blocco temporaneo che impedisce ulteriori tentativi di accesso per un periodo di due minuti.

```
failed_key = f"failed_login_{email}"
block_key = f"blocked_login_{email}"

if cache.get(block_key):
    errore = "Troppi tentativi falliti. Riprova tra 2 minuti."

failed_attempts = cache.get(failed_key, 0) + 1

if failed_attempts >= 3:
    cache.set(block_key, True, timeout=120)
```

La contromisura è stata implementata tramite un sistema di caching che memorizza il numero di tentativi falliti associati a ciascun account. Al raggiungimento della soglia prefissata viene

creato un blocco temporaneo della durata di 120 secondi, riducendo significativamente l'efficacia degli attacchi a forza bruta.

7.3 Attacco a dizionario

L'attacco a dizionario consiste nel tentare l'accesso a un sistema utilizzando liste predefinite di password comuni, come "123456", "password", "qwerty" o altre combinazioni frequentemente utilizzate dagli utenti. A differenza dell'attacco a forza bruta, che prova tutte le combinazioni possibili, l'attacco a dizionario si concentra sulle password statisticamente più diffuse, risultando spesso più rapido ed efficace.

Per ridurre il rischio associato a questo tipo di attacco, nel progetto AutoGarage sono state adottate diverse contromisure, tra cui l'utilizzo di password complesse durante la registrazione e la memorizzazione delle credenziali in forma hashata all'interno del database.

```
if len(password) < 8:
    raise forms.ValidationError("La password deve contenere almeno 8 caratteri.")

if nome and nome in password_lower:
    raise forms.ValidationError("La password non può contenere il nome dell'utente.")

if cognome and cognome in password_lower:
    raise forms.ValidationError("La password non può contenere il cognome dell'utente.")

if parte_email and parte_email in password_lower:
    raise forms.ValidationError("La password non può contenere la parte iniziale dell'email.")

if not re.search(r"[A-Z]", password):
    raise forms.ValidationError("La password deve contenere almeno una lettera maiuscola.")

if not re.search(r"[a-z]", password):
    raise forms.ValidationError("La password deve contenere almeno una lettera minuscola.")

if not re.search(r"\d", password):
    raise forms.ValidationError("La password deve contenere almeno un numero.")

if not re.search(r"[^\w\s]", password):
    raise forms.ValidationError("La password deve contenere almeno un carattere speciale.")
```

Durante la registrazione il sistema verifica che la password soddisfi specifici requisiti di sicurezza. In particolare, deve contenere almeno otto caratteri, una lettera maiuscola, una lettera minuscola, un numero e un carattere speciale. Questi controlli riducono la probabilità che vengano utilizzate password facilmente individuabili tramite attacchi a dizionario.

```
utente.password = make_password(self.cleaned_data["password"])
```

Le password non vengono memorizzate in chiaro nel database. Prima del salvataggio viene applicata la funzione `make_password()`, che genera un hash della password. In questo modo, anche in caso di accesso non autorizzato al database, le credenziali originali non risultano direttamente leggibili.

- La password deve contenere almeno una lettera maiuscola.

Password:

Conferma password:

Nel caso mostrato, il sistema rifiuta la password inserita perché non rispetta uno dei requisiti minimi di sicurezza previsti. Anche se viene visualizzato un solo errore alla volta, la password deve comunque soddisfare tutti i controlli definiti nel form di registrazione.

8 Conclusioni

Il progetto AutoGarage ha permesso di applicare concretamente le principali metodologie di progettazione e sviluppo di basi di dati studiate durante il corso. A partire dall'analisi dei requisiti sono stati realizzati il modello E-R, il modello logico e il database relazionale, successivamente integrati all'interno di un'applicazione web sviluppata con Django e MySQL.

L'applicazione implementa le principali funzionalità richieste per la gestione di una concessionaria automobilistica, tra cui la registrazione e l'autenticazione degli utenti, la consultazione del catalogo auto, la prenotazione di test drive, la visualizzazione delle recensioni e l'accesso a dashboard dedicate ai diversi ruoli del sistema.

Particolare attenzione è stata inoltre dedicata agli aspetti di sicurezza. Sono state adottate contromisure contro attacchi di SQL Injection, forza bruta e dizionario, oltre a meccanismi di validazione degli input, gestione sicura delle password e controllo degli accessi basato sui ruoli. Tali soluzioni contribuiscono a garantire l'affidabilità dell'applicazione e la protezione dei dati memorizzati.

Nel complesso, il progetto ha consentito di integrare aspetti teorici e pratici della progettazione di basi di dati, fornendo un sistema informativo funzionante, coerente con i requisiti iniziali e facilmente estendibile con ulteriori funzionalità.